NEJVĚTŠÍ ČESKO-SLOVENSKÁ KONFERENCE O IT

# TechEd

Get **Inspired**

19. - 21. 5. 2026 | Praha / ONLINE

# Zlatá pravidla designu REST API

Miroslav Holec | TechEd 2026

mirek@miroslavholec.cz

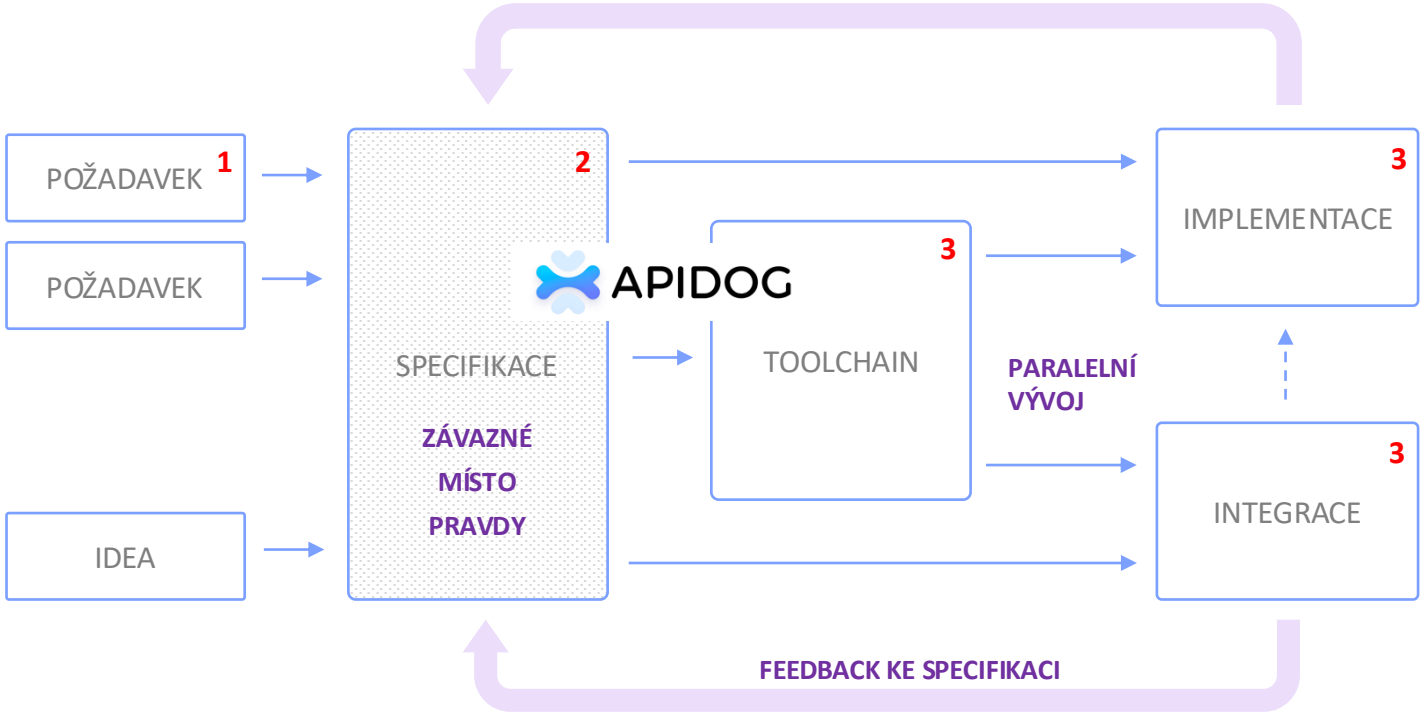
restapi.cz



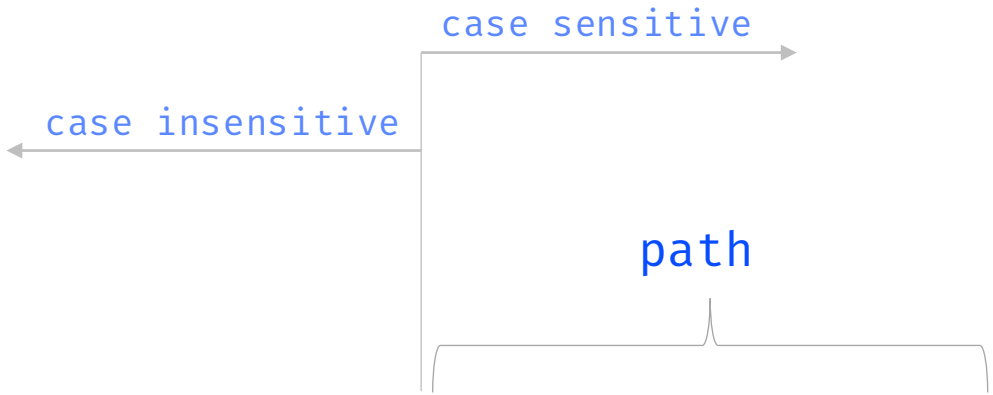

TechEd GOPAS

# Design REST API

- REST neříká nic o tom, jestli je vaše API kvalitní
- návrh API a tvorba specifikace je proces, který předchází implementaci



# Struktura URL




https://**api**.website.cz/public/v1/customers?page=1&name=john

**auth**.website.cz

**developer**.website.cz

**developer**.website.cz/docs

**developer**.website.cz/docs/reference

# Resource-based URL design

- vychází z doménového modelu aplikace
- doménové entity jsou vystavené jako resources
- URL jednoznačně identifikuje množinu dat a je unikátní
- používáme **podstatné jméno v množném čísle**

~~`user/{id}`~~
~~`users/invoices`~~
~~`users/{id}/name`~~
~~`users/{id}/disable`~~

**`resource1`**`/{id1}/`**`resource2`**`/{id2}/`**`resource3`**`/{id3}`

| kolekce dat | instance |
| --- | --- |
| **`users`** | **`users`**`/{id}` |

# URL design patterns

- pro filtrování používáme query parameters

~~`customers/{cid}/invoices`~~

~~`orders/{oid}/invoices`~~

`invoices?orderId={oid}&customerId={cid}`

- vztah 1:1 modelujeme jako pseudoresource

~~`users/{id}/profile`~~

~~`users/{id}/image`~~

`user-profiles/{id}`

`user-images/{id}`

# Identifikátory resources

- identifikátor resource je umístěn vždy v url path
- API vrací vždy 0 nebo 1 záznam jako objekt

~~`customers?id={id}`~~ `customers/{id}`

- jaký zvolit typ identifikátoru?

~~`users/198`~~

~~`users/mirek@miroslavholec.cz`~~

`users/f8ed6fc2-905c-4c31-8159-641027e259b0`

`users/p8am21sp`

# Query Design

- zahrnuje řazení, stránkování a filtrování
- implicitní pravidlo [ prvních 10 položek, neodstraněných, seřazených dle ID ]
- standardní validace vstupů [ povolené řazení, max items per page, … ]
- rovnítko = rovná se

```
products?categoryId=45&location=Prague&published=2025-01-01
products?categoryId=45&locationCt=Prague&publishedFrom=2025-01-01
```

- není prostor pro kreativitu

```
products?expand=categories
products?fields=id,name
```



# Unified resource pro GET a P-metody

- jednotný resource pro GET by ID a odpověď na PUT, POST, PATCH
- neobsahuje nested data (efektivita cachování)

```
{
   "id" : "p8ukh54ea",
   "title" : "Můj článek",
   "authorId" : jn67ajs
}
```

```
{
   "title" : "Můj článek",
   "author" : {
      "name" : "Jan Novak"
   }
}
```

```
{
  "article" : {
     "id" : 1,
     "title" : "Můj článek",
     "author" : "Jan Novak"
   }
}
```

# Každá P-metoda má svůj účel

| metoda | operace | body | safe | idemp. | cache | status code |
|---|---|---|---|---|---|---|
| POST | vytvoření *(čtení)* | ano | ne | **ne** | *může* | 201 |
| PATCH | částečný update | ano | ne | **ne** | ne | 200 |
| PUT | replace *(vytvoření)* | ano | ne | ano | ne | 200, 201 |

- při vytvoření resource je vhodné vrátit `Location` header
- všechny metody vrací stejnou strukturu jako GET by ID

```
{
   "id" : "p8ukh54ea",
   "title" : "Můj článek",
   "author" : jn67ajs
}
```

# Non-CRUD jako POST pseudoresource

- při modelování interakcí použijeme **resources** místo názvů událostí

~~**invoices**/{id}/paid~~ **payments** `{ "vs" : 202656543, "total" : 9999 }`

~~**articles**/{id}/publish~~ **releases** `{ "articleId" : "k9hjs3a" }`

~~**players**/{id}/play~~ **player-events**

~~**accounts**/{id}/activate~~ **account-activations**

# Content Negotiation

- konkrétní formát odpovědi lze řešit pomocí HTTP hlaviček
- mediatype v URL používáme jen u statických assetů (css, js, webp)

~~`invoices/{id}/export`~~ `invoices/{id}` + `Accept: text/xml`

~~`invoices/{id}.xml`~~

- hlavičky: Accept, Accept-Language
- status 406: Not Acceptable

# HATEOAS

- Hypermedia as the Engine of Application State (známe jako `<a href="">`)
- v REST API není lidská interaktivita, nepotřebujeme linkování resources

```
{
   "id" : "p8ukh54ea",
   "title" : "Můj článek",
   "author" : "Jan Novak",
   "_links" : {
      "author" : {
         "href" : "/authors/78"
      }
   }
}
```

```
{
   "id" : "p8ukh54ea",
   "title" : "Můj článek",
   "authorId" : 78
}
```

# Konvence v kontraktech

- JSON je notace pro JS, používat vždy camelCase
- zvolit si standardy a použít je pro pokrytí hodnot, např.: `cz,cs,sk` dle ISO 639-1
- smysluplné názvy vlastností, např.: `isActive, createdAt, documentType`
- konzistence napříč rozhraním: `startDate, dateFrom, dateStart`
- objekty místo primitiv: `clientName, clientStreet, clientZip, clientCountry`

```
{
   "id" : "p8ukh54ea",
   "documentType" : "invoice",
   "status" : "active",
   "client" : {
      "name" : "Jan Novak",
      "street" : "Novakova 87/2"
```

# Chyby a chybové struktury jednotně

- všechny chybové stavy `4xx` a `5xx` používají stejnou chybovou strukturu
- lze použít RFC 9457 (v .NET jako ProblemDetails)
- co má prioritu když „*update neexistující ID, nemá oprávnění, nevalidní struktura, chybný accept*“

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json
{
   "type": "https://restapi.cz/probs/problem-details",
   "title": "You do not have enough credit",
   "detail": "Your balance is 30, but that costs 50",
   "status": 403,
   "errors": []
```

# Neverzování

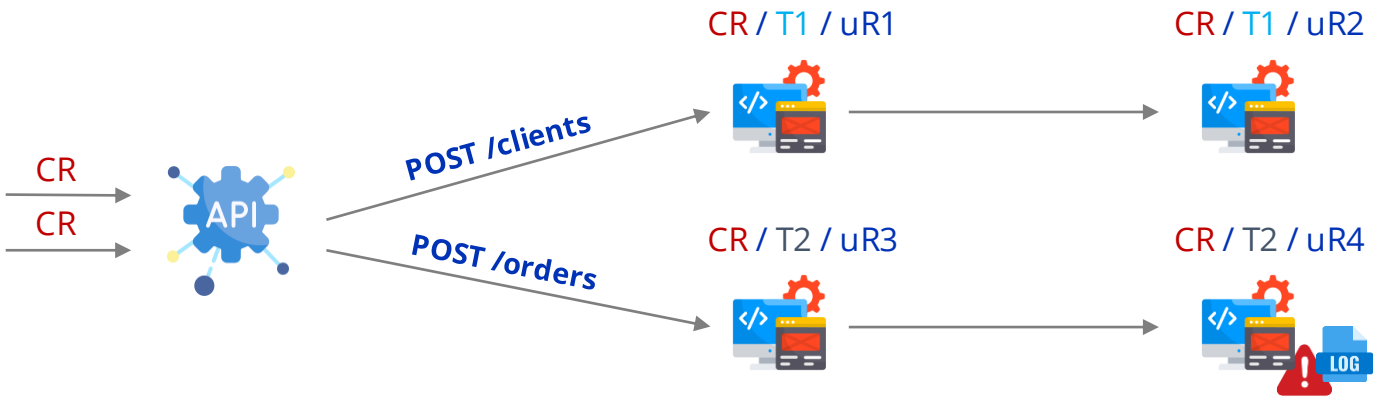
- cílem je neverzovat – předejít verzování – oddálit verzování
- zvažovat potenciální změny v rozhraní, volit raději overengineering
- u klientů jako mobilní aplikace a desktop vynucovat aktualizace
- dělat zpětně kompatibilní změny
- dělat zpětně nekompatibilní změny, ale komunikovat je

```
https://api.website.cz/v1/users
https://api.website.cz/v2/users
```

# Logging

- každý požadavek má přidělen specifický a unikátní identifikátor
- při vzniku chyby dojde k zalogování včetně identifikátoru a vrácení identifikátoru na klienta
- RequestID si generuje server sám, ostatní může přijmout od klienta

| | |
|---|---|
| **RequestId** | unikátní pro každý samostatný HTTP požadavek každé služby |
| **TraceId** | identifikuje řetěz (trasu) požadavků |
| **CorrelationId** | předává se mezi HTTP požadavky pro vysledování komunikace mezi službami |



# Závěr

- dobrá API vznikají na základě návrhu
- vyzkoušejte ApiDog pro specifikaci, mockování a testování API
- seznamte se s principy moderních REST API



mirek@miroslavholec.cz

**restapi.cz**


TechEd
Get Inspired